

로지스틱회귀

"그래서 될까, 안 될까?"를 **확률로** 대답해주는 기계.
이름에 '회귀'가 붙어 있지만 사실은 분류 모델입니다.

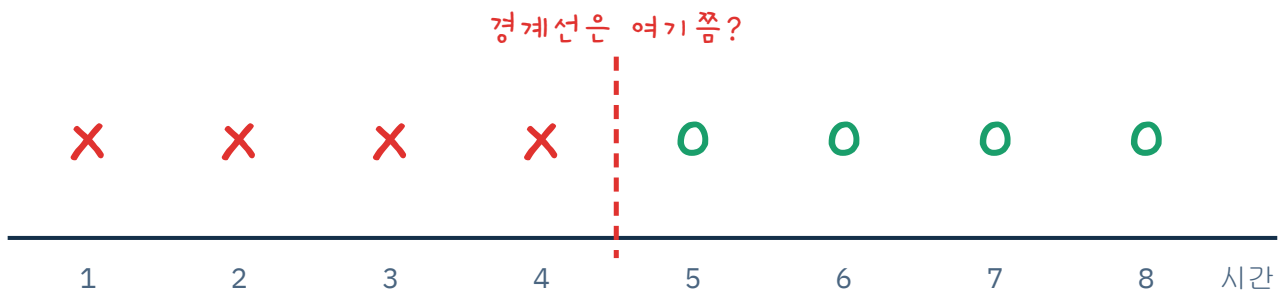
Q1 이걸 뭐 하는 애냐면

"이 문자 스팸이야?" → 스팸일 확률 92%
"내일 비 올까?" → 올 확률 30%
"나 시험 붙을까?" → 합격 확률 78%

답이 딱 두 개(예/아니오)인 질문에, 그냥 찍지 않고 몇 퍼센트인지 까지 알려주는 게 이 모델의 일입니다.

Q2 이야기로 시작해봅시다

반 친구 8명의 공부시간과 시험 결과입니다.

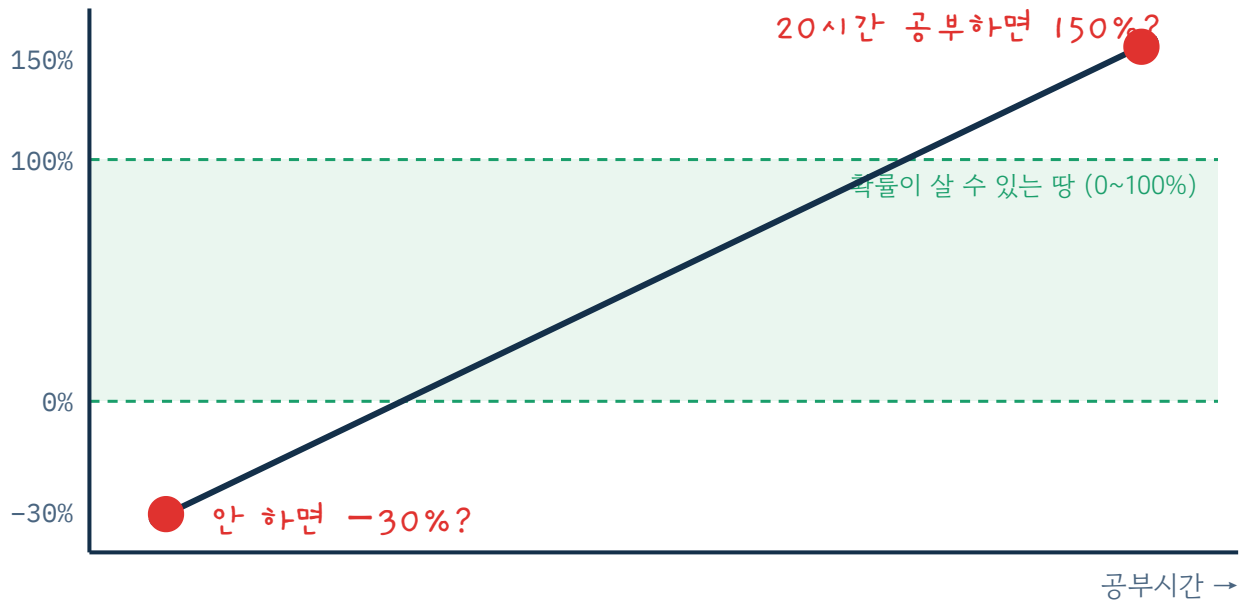


딱 봐도 보이죠? 4시간과 5시간 사이 어딘가에 경계가 있습니다.

그럼 4.5시간 공부한 친구는요? "몰라, 반반쯤?"
이 "반반쯤"을 숫자로 정확히 계산해주는 게 로지스틱 회귀입니다.

Q3 왜 직선으로는 안 될까

"공부시간이 늘면 합격률도 오르니까 그냥 직선 그으면 되잖아?"
그런데 직선을 쭉 그으면 이런 일이 벌어집니다.

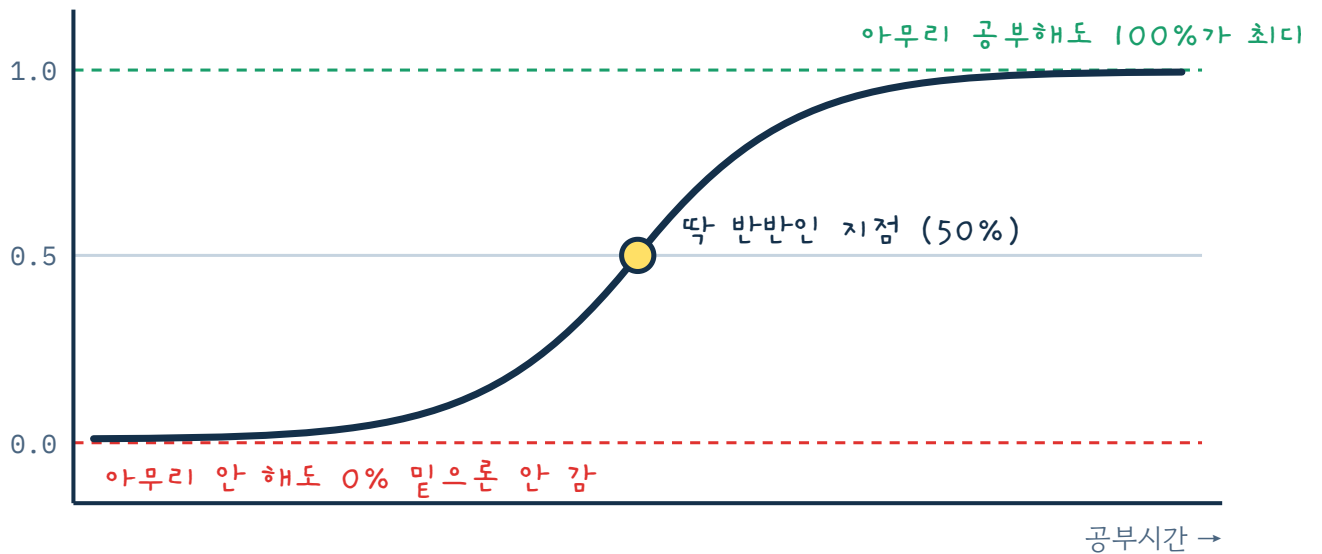


직선은 위아래로 끝없이 뻗어나갑니다. 그래서 확률로 쓸 수 없어요.

확률이 150%, -30%가 나옵니다. 말이 안 되죠. 확률은 0%에서 100% 사이여야 하니까요.

Q4 그래서 직선을 S자로 구부립니다

직선의 양 끝을 꼭 눌러서 0과 1에 딱 붙게 만듭니다.



이 S자 곡선을 **시그모이드(sigmoid)**라고 부릅니다. 이름의 S가 곧 모양이에요.

하는 일은 딱 하나입니다. 어떤 숫자가 들어와도 0~1 사이로 찌그러뜨리기.

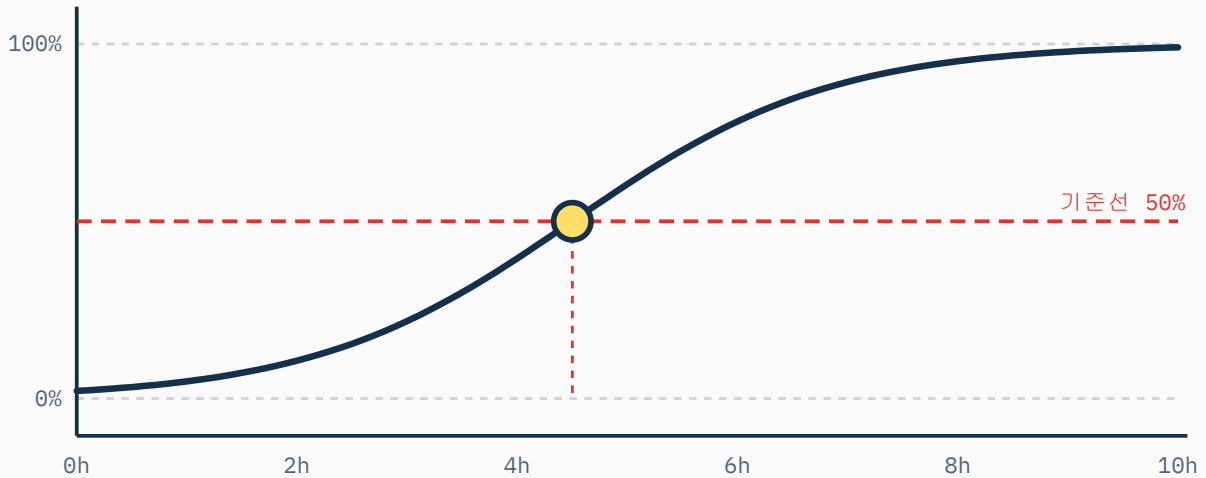
넣은 값	나온 값	
-100	0.000...	아무리 작아도 0 밑으론 안 감
-2	0.12	
0	0.50	정확히 반반
2	0.88	
100	0.999...	아무리 커도 1을 안 넘음

그래서 안심하고 "**확률**"이라고 부를 수 있게 되죠.

Q5 직접 움직여보기

공부시간 → 합격확률 계산기

막대를 움직이면 곡선 위의 점과 확률이 같이 움직입니다.



합격 확률

50%

합격

공부시간

4.5시간

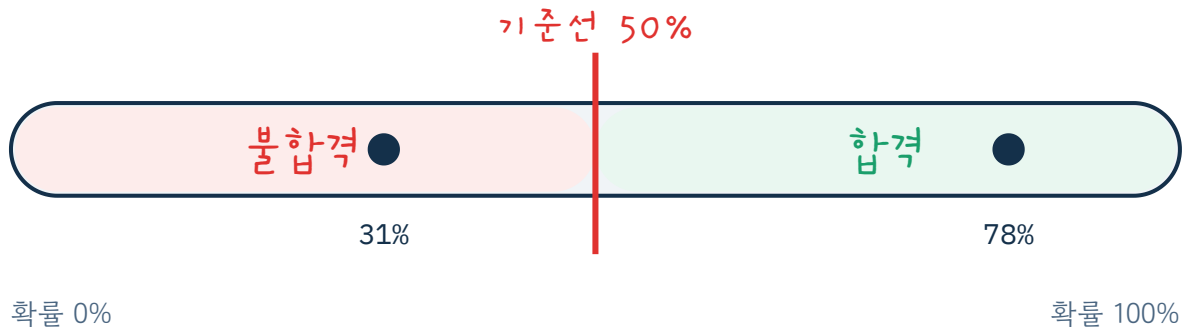
합격이라고 부를 기준선

50%

기준선을 20%까지 내려보세요. 2시간만 공부해도 "합격"이라고 판정합니다. 같은 모델인데 성격이 완전히 달라지죠.

Q6 확률을 답으로 바꾸기

모델이 "합격 확률 78%"라고 하면, 마지막에 **기준선을 그어서** 자릅니다.



이 50%는 고정입니다. 상황에 따라 바뀌요.

- 📧 **암 검사** → 기준을 **20%**로 낮춤. 조금만 의심스러워도 "더 검사해보자"고 해야 안전하니까
- 🗑️ **스팸 필터** → 기준을 **90%**로 올림. 중요한 메일이 스팸함에 처박히면 큰일이니까

Q7 코드로 해보기

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# 공부시간과 결과 (0=불합격, 1=합격)
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8]])
y = np.array([ 0, 0, 0, 0, 1, 1, 1, 1 ])

model = LogisticRegression()
model.fit(X, y) # ← 이 한 줄이 "공부하는" 부분

# 시간별로 합격 확률 물어보기
for h in [1, 3, 4.5, 6, 8]:
    p = model.predict_proba([[h]])[0][1]
    print(f"{h}시간 공부 → 합격확률 {p:.0%}")
```

```
1시간 공부 → 합격확률 6%
3시간 공부 → 합격확률 25%
4.5시간 공부 → 합격확률 50% ← 딱 반반!
6시간 공부 → 합격확률 81%
8시간 공부 → 합격확률 96%
```

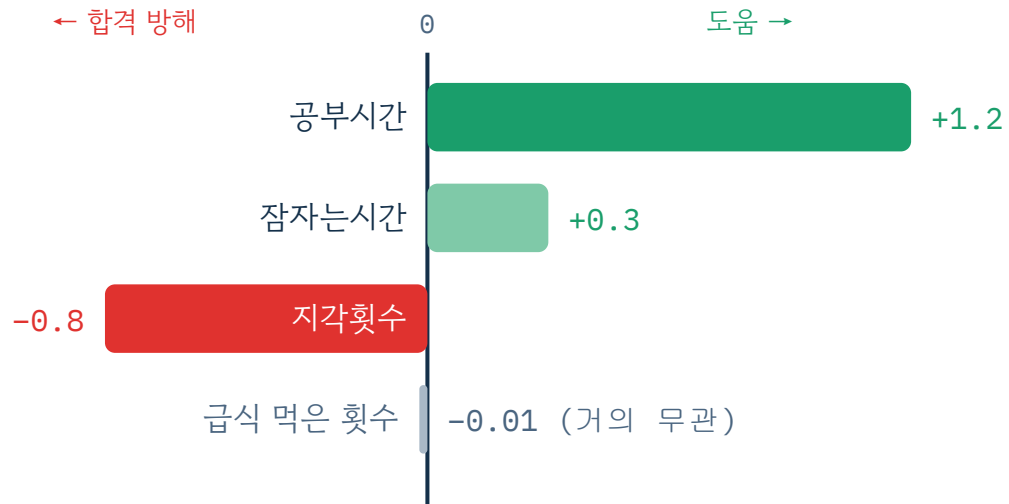
앞에서 눈으로 "4~5시간 사이가 경계"라고 봤던 걸, 모델이 **4.5시간**이라고 정확히 짚어줬습니다.

꼭 기억할 것 하나

`predict` 는 "합격!"만 알려주고, `predict_proba` 는 **얼마나 확신하는지** 까지 알려줍니다. 51%로 겨우 합격인지, 99%로 확실한 합격인지는 완전히 다른 정보죠. 실무에선 이쪽을 훨씬 많이 씁니다.

Q8 진짜 매력 — 이유를 말해줍니다

여러 정보를 같이 넣으면, 모델이 뭐가 중요했는지 숫자로 알려줍니다.



양수면 도움, 음수면 방해, 0에 가까우면 상관없음. 막대가 길수록 영향력이 큼.

이게 왜 중요하냐면 — 요즘 유행하는 딥러닝은 성능은 좋은데 "왜 그렇게 판단했는지" 설명을 못 합니다. 은행에서 대출을 거절당했는데 "AI가 안 된대요"라고만 하면 곤란하잖아요. 그래서 이유를 설명해야 하는 분야(은행, 병원)에서는 로지스틱 회귀가 아직도 현역입니다.

Q9 정리

	선형 회귀	로지스틱 회귀
답하는 것	숫자 (집값 3억, 기온 27도)	확률 → 예/아니오
생긴 모양	직선 /	S자 곡선
종류	회귀	분류

좋은 점 — 빠르고, 가볍고, 이유를 설명해줌. 새 문제를 만나면 일단 이걸로 먼저 해보고 기준점을 잡습니다.

아쉬운 점 — 기본적으로 선 하나로 나누는 애라서, 복잡하게 뒤엉킨 데이터는 잘 못 나눕니다.

개 / 고양이 / 새처럼 셋 이상을 나누고 싶다면? y 에 0, 1, 2를 넣기만 하면 사이킷런이 알아서 처리해줍니다. 각각의 확률(개 70%, 고양이 25%, 새 5%)을 계산해서 제일 높은 걸 고르는 방식이에요.

Q10 Level별 코드 작성하기

옆에 주피터 노트북을 열고, 복사 버튼으로 코드를 붙여넣어 따라 해보세요. 쉬운 것부터 실전(캐글 공공데이터)까지 3단계입니다.

코드 해석 ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

LEVEL 1 단순 — 유방암 확률 분류

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
model = LogisticRegression(max_iter=5000).fit(X_tr, y_tr)
print('정확도:', round(accuracy_score(y_te, model.predict(X_te)), 3))
```

`load_breast_cancer(return_X_y=True)`

세포 측정치로 악성/양성을 맞추는 데이터.

`LogisticRegression().fit`

로지스틱 회귀로 학습(확률로 분류).

`accuracy_score`

정확도로 성능 확인.

LEVEL 2 복잡 — 스케일링 + 계수(coef) 해석

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(data.data, data.target, test_size=0.2, random_state=0,
stratify=data.target)
sc = StandardScaler()
X_tr_s = sc.fit_transform(X_tr)
model = LogisticRegression(max_iter=5000).fit(X_tr_s, y_tr)

coef = pd.Series(model.coef_[0], index=data.feature_names).sort_values()
print('계수 낮은(작을수록 악성 쪽) 3개:'); print(coef.head(3).round(2))
print('계수 높은(클수록 양성 쪽) 3개:'); print(coef.tail(3).round(2))
```

StandardScaler + fit

계수를 공정하게 비교하려면 같은 잣대가 필수.

model.coef_[0]

각 특성의 '영향력 점수'. 부호=방향, 크기=세기.

coef.head/tail

계수가 낮은/높은 특성을 뽑아 '무엇이 판단을 갈랐나'를 읽습니다.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
df = pd.read_csv(url)
df['Age'] = df['Age'].fillna(df['Age'].median())
X = pd.get_dummies(df[['Pclass', 'Sex', 'Age', 'Fare']], drop_first=True)
y = df['Survived']

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
sc = StandardScaler()
X_tr_s = sc.fit_transform(X_tr)
X_te_s = sc.transform(X_te)
model = LogisticRegression(max_iter=5000).fit(X_tr_s, y_tr)
print('정확도:', round(accuracy_score(y_te, model.predict(X_te_s)), 3))
print('계수(양수=생존확률 UP):'); print(pd.Series(model.coef_[0], index=X.columns).round(2))
```

```
get_dummies(..., drop_first=True)
```

성별을 0/1 한 열로(중복 방지).

```
StandardScaler + LogisticRegression
```

스케일 후 학습 -> 계수 비교가 공정.

```
model.coef_[0]
```

양수면 생존 확률을 올리는 특성(예: 여성).